Write a C program to receive bytes of data serially continuously and put them into PORTB. Use interrupt method. Fosc = 8MHz, baud rate (br) = 7200.

X = fosc/(16 br) -1 = 68.4 use 68 UBRRL = 68

Vector address for USART, Receive Complete (RXC) 0x001A

ISR string: USART0_RX_vect

```
#include <avr/io.h>
#include <avr/interrupt.h>
ISR(USART0_RX_vect);
INT MAIN() {
        DDRB = 0XFF; // PB OUTPUT
        UCSRB = (1<<RXEN) | (1<<RXCIE); // ENABLE RECEPTION + RXC INTERRUPT SOURCE (UCSZ2 = 0)
        UCSRC = (1<<UCSZ1) | (1<<UCSZ0) | (1<<URSEL); // 8 BITS, 1 STOP, NO PARITY, ENABLE UCSRC
        UBRRL = 68;  // 7200 BPS
        // NOTE THAT U2X IN UCSRA IS 0, MEANING 16 USED
        SEI(); // I = 1 GLOBAL INTERRUPT ENABLE
        WHILE(1);
        RETURN 0;
}
ISR(USART0_RX_vect) {
        PORTB = UDR; // RECEIVED SERIAL DATA IS IN UDR TO BE READ AND OUTPUT TO PB
}
```

Write a C program to transmit ‘b’ serially continuously. Use interrupt method and double asynchronous mode. Fosc = 8MHz, baud rate (br) = 7200.

X = fosc/(8 br) -1 = 137.9 use 138 UBRRL = 138 (U2X = 1 IN UCSRA)

Vector address for USART, Data Register Empty (UDRE) 0x001C

ISR string: USART0_UDRE_vect

```
#include <avr/io.h>
#include <avr/interrupt.h>
ISR(USART0_UDRE_vect);
INT MAIN() {
        UCSRA = (1<<U2X);  // DOUBLE BAUD RATE (DOUBLE ASYNCHRONOUS MODE)
        UCSRB = (1<<TXEN) | (1<<UDRIE); // ENABLE TRANSMISSION + UDRE INTERRUPT SOURCE (UCSZ2 = 0)
        UCSRC = (1<<UCSZ1) | (1<<UCSZ0) | (1<<URSEL); // 8 BITS, 1 STOP, NO PARITY, ENABLE UCSRC
        UBRRL = 138;  // 7200 BPS
        SEI(); // I = 1 GLOBAL INTERRUPT ENABLE
        WHILE(1);
        RETURN 0;
}
ISR(USART0_UDRE_vect) {
        UDR = ‘b’; // TRANSMISSION BEGINS AFTER UDRE FLAG = 1 AND UDR IS ASSIGNED THE VALUE ‘b’
}
```

Convert the C program to assembly to receive bytes of data serially continuously and put them into PORTB. Use interrupt method. Fosc = 8MHz, baud rate (br) = 7200.

X = fosc/(16 br) -1 = 68.4 use 68 UBRRL = 68

Vector address for USART, Receive Complete (RXC) 0x001A

ISR string: USART0_RX_vect

```
#include <avr/io.h>
#include <avr/interrupt.h>
ISR(USART0_RX_vect);
INT MAIN() {
        DDRB = 0XFF; // PB OUTPUT
        UCSRB = (1<<RXEN) | (1<<RXCIE); // ENABLE RECEPTION + RXC INTERRUPT SOURCE (UCSZ2 = 0)
        UCSRC = (1<<UCSZ1) | (1<<UCSZ0) | (1<<URSEL); // 8 BITS, 1 STOP, NO PARITY, ENABLE UCSRC
        UBRRL = 68;  // 7200 BPS
        // NOTE THAT U2X IN UCSRA IS 0, MEANING 16 USED
        SEI(); // I = 1 GLOBAL INTERRUPT ENABLE
        WHILE(1);
        RETURN 0;
}
ISR(USART0_RX_vect) {
        PORTB = UDR; // RECEIVED SERIAL DATA IS IN UDR TO BE READ AND OUTPUT TO PB
}
```

Assembly

```
.ORG 0
RJMP MAIN
.ORG 0X001A
RJMP ISR_RXC
.ORG 0X50
MAIN:  LDI R16, 0XFF
        OUT DDRB, R16; DDRB = 0XFF
        LDI R16, (1<<RXEN) | (1<<RXCIE); // ENABLE RECEPTION + RXC INTERRUPT SOURCE (UCSZ2 = 0)
        OUT UCSRB, R16;
        LDI R16, (1<<UCSZ1) | (1<<UCSZ0) | (1<<URSEL); // 8 BITS, 1 STOP, NO PARITY, ENABLE UCSRC
        OUT UCSRC, R16;
        LDI R16, 68; UBRRL = 68 FOR 7200 BPS
        OUT UBRRL, R16
        // NOTE THAT U2X IN UCSRA IS 0, MEANING 16 USED
        SEI; // I = 1 GLOBAL INTERRUPT ENABLE
        HERE:  RJMP HERE; CONTINUOUS LOOP WAITING FOR SERIALLY RECEIVED DATA
.ORG 0X75
ISR_RXC:        IN R17, UDR; UDR HAS RECEIVED BYTE
                OUT PORTB, R17; COPY RECEIVED BYTE TO PB
                RETI
```

Convert the C program to assembly to transmit 'b' serially continuously. Use interrupt method and double asynchronous mode. Fosc = 8MHz, baud rate (br) = 7200.

X = fosc/(8 br) -1 = 137.9 use 138 UBRRL = 138 (U2X = 1 IN UCSRA)

Vector address for USART, Data Register Empty (UDRE) 0x001C

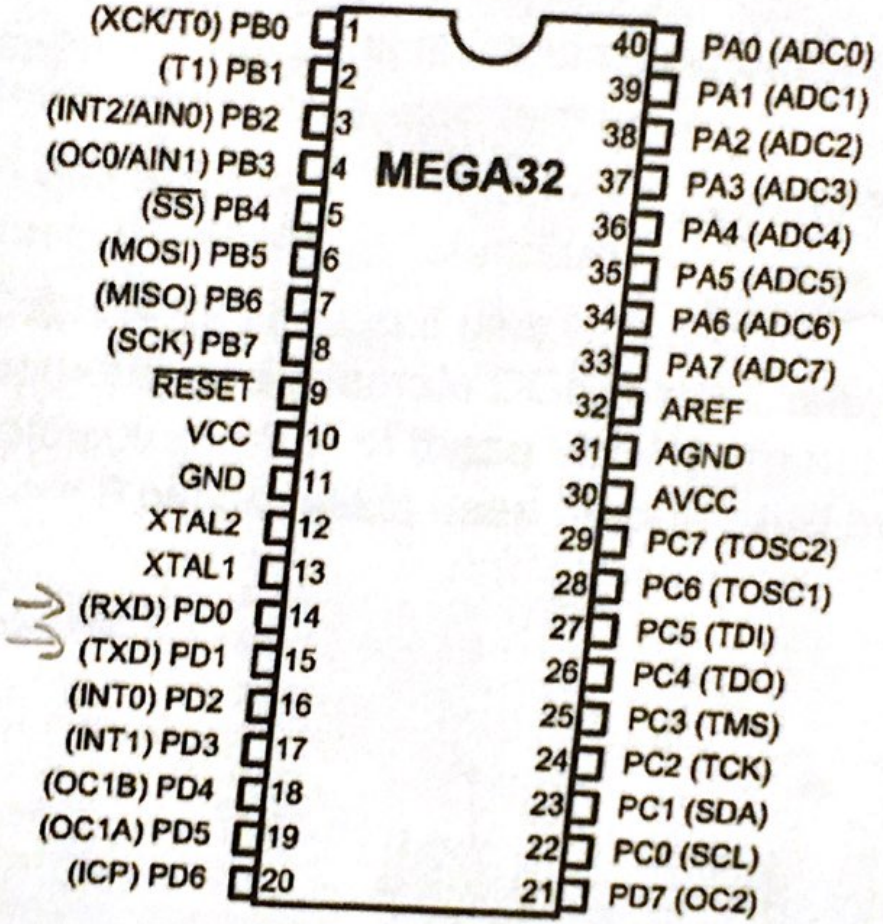
ISR string: USART0_UDRE_vect

```
#include <avr/io.h>
#include <avr/interrupt.h>
ISR(USART0_UDRE_vect);
INT MAIN() {
        UCSRA = (1<<U2X);  // DOUBLE BAUD RATE (DOUBLE ASYNCHRONOUS MODE)
        UCSRB = (1<<TXEN) | (1<<UDRIE); // ENABLE TRANSMISSION + UDRE INTERRUPT SOURCE (UCSZ2 = 0)
        UCSRC = (1<<UCSZ1) | (1<<UCSZ0) | (1<<URSEL); // 8 BITS, 1 STOP, NO PARITY, ENABLE UCSRC
        UBRRL = 138;  // 7200 BPS
        SEI(); // I = 1 GLOBAL INTERRUPT ENABLE
        WHILE(1);
        RETURN 0;
}
ISR(USART0_UDRE_vect) {
        UDR = 'b'; // TRANSMISSION BEGINS AFTER UDRE FLAG = 1 AND UDR IS ASSIGNED THE VALUE 'b'
}
```

ASSEMBLY

```
.ORG 0
RJMP MAIN
.ORG 0X001C
RJMP ISR_UDRE
.ORG 0X50
MAIN:  LDI R18, 'b'; 'b' FOR SERIAL TRANSMISSION
        LDI R16, 1<<U2X; // U2X = 1 FOR DOUBLE BAUD RATE
        OUT UCSRA, R16;
        LDI R16, (1<<TXEN) | (1<<UDRIE); // ENABLE TRANSMISSION + UDRE INTERRUPT SOURCE (UCSZ2 = 0)
        OUT UCSRB, R16;
        LDI R16, (1<<UCSZ1) | (1<<UCSZ0) | (1<<URSEL); // 8 BITS, 1 STOP, NO PARITY, ENABLE UCSRC
        OUT UCSRC, R16;
        LDI R16, 138; UBRRL = 138 FOR 7200 BPS
        OUT UBRRL, R16
        SEI; // I = 1 GLOBAL INTERRUPT ENABLE
        HERE:  RJMP HERE; SERIAL 'b' TO BE TRANSMITTED CONTINUOUSLY THROUGH INTERRUPT
.ORG 0X75
ISR_UDRE:      OUT UDR, R18; TRANSMIT 'b' WHEN UDRE = 1
                RETI
```

Chapter 4-Fig4-1



Address		Name
Mem.	I/O	
$20	$00	TWBR
$21	$01	TWSR
$22	$02	TWAR
$23	$03	TWDR
$24	$04	ADCL
$25	$05	ADCH
$26	$06	ADCSRA
$27	$07	ADMUX
$28	$08	ACSR
$29	$09	UBRRL
$2A	$0A	UCSRB
$2B	$0B	UCSRA
$2C	$0C	UDR
$2D	$0D	SPCR
$2E	$0E	SPSR
$2F	$0F	SPDR
$30	$10	PIND
$31	$11	DDRD
$32	$12	PORTD
$33	$13	PINC
$34	$14	DDRC
$35	$15	PORTC

Address		Name
Mem.	I/O	
$36	$16	PINB
$37	$17	DDRB
$38	$18	PORTB
$39	$19	PINA
$3A	$1A	DDRA
$3B	$1B	PORTA
$3C	$1C	EECR
$3D	$1D	EEDR
$3E	$1E	EEARL
$3F	$1F	EEARH
$40	$20	UBRRC
		UBRRH
$41	$21	WDTCR
$42	$22	ASSR
$43	$23	OCR2
$44	$24	TCNT2
$45	$25	TCCR2
$46	$26	ICR1L
$47	$27	ICR1H
$48	$28	OCR1BL
$49	$29	OCR1BH
$4A	$2A	OCR1AL

Address		Name
Mem.	I/O	
$4B	$2B	OCR1AH
$4C	$2C	TCNT1L
$4D	$2D	TCNT1H
$4E	$2E	TCCR1B
$4F	$2F	TCCR1A
$50	$30	SFIOR
$51	$31	OCDR
		OSCCAL
$52	$32	TCNT0
$53	$33	TCCR0
$54	$34	MCUCSR
$55	$35	MCUCR
$56	$36	TWCR
$57	$37	SPMCR
$58	$38	TIFR
$59	$39	TIMSK
$5A	$3A	GIFR
$5B	$3B	GICR
$5C	$3C	OCR0
$5D	$3D	SPL
$5E	$3E	SPH
$5F	$3F	SREG

Note: Although memory address $20-$5F is set aside for I /O registers (SFR) we can access them as I /O locations with addresses starting at $00.

Figure 2-7. I/O Registers of the ATmega32 and Their Data Memory Address Locations

RXC	TXC	UDRE	FE	DOR	PE	U2X	MPCM

RXC (Bit 7): USART Receive Complete
This flag bit is set when there are new data in the receive buffer that are not read yet. It is cleared when the receive buffer is empty. It also can be used to generate a receive complete interrupt.

TXC (Bit 6): USART Transmit Complete
This flag bit is set when the entire frame in the transmit shift register has been transmitted and there are no new data available in the transmit data buffer register (TXB). It can be cleared by writing a one to its bit location. Also it is automatically cleared when a transmit complete interrupt is executed. It can be used to generate a transmit complete interrupt.

UDRE (Bit 5): USART Data Register Empty
This flag is set when the transmit data buffer is empty and it is ready to receive new data. If this bit is cleared you should not write to UDR because it overrides your last data. The UDRE flag can generate a data register empty interrupt.

FE (Bit 4): Frame Error
This bit is set if a frame error has occurred in receiving the next character in the receive buffer. A frame error is detected when the first stop bit of the next character in the receive buffer is zero.

DOR (Bit 3): Data OverRun
This bit is set if a data overrun is detected. A data overrun occurs when the receive data buffer and receive shift register are full, and a new start bit is detected.

PE (Bit 2): Parity Error
This bit is set if parity checking was enabled (UPM1 = 1) and the next character in the receive buffer had a parity error when received.

U2X (Bit 1): Double the USART Transmission Speed
Setting this bit will double the transfer rate for asynchronous communication.

MPCM (Bit 0): Multi-processor Communication Mode
This bit enables the multi-processor communication mode. The MPCM feature is not discussed in this book.

Notice that FE, DOR, and PE are valid until the receive buffer (UDR) is read. Always set these bits to zero when writing to UCSRA.

Figure 11-13. UCSRA: USART Control and Status Register A

RXCIE	TXCIE	UDRIE	RXEN	TXEN	UCSZ2	RXB8	TXB8

RXCIE (Bit 7): Receive Complete Interrupt Enable
To enable the interrupt on the RXC flag in UCSRA you should set this bit to one.

TXCIE (Bit 6): Transmit Complete Interrupt Enable
To enable the interrupt on the TXC flag in UCSRA you should set this bit to one.

UDRIE (Bit 5): USART Data Register Empty Interrupt Enable
To enable the interrupt on the UDRE flag in UCSRA you should set this bit to one.

RXEN (Bit 4): Receive Enable
To enable the USART receiver you should set this bit to one.

TXEN (Bit 3): Transmit Enable
To enable the USART transmitter you should set this bit to one.

UCSZ2 (Bit 2): Character Size
This bit combined with the UCSZ1:0 bits in UCSRC sets the number of data bits (character size) in a frame.

RXB8 (Bit 1): Receive data bit 8
This is the ninth data bit of the received character when using serial frames with nine data bits. This bit is not used in this book.

TXB8 (Bit 0): Transmit data bit 8
This is the ninth data bit of the transmitted character when using serial frames with nine data bits. This bit is not used in this book.

Figure 11-14. UCSRB: USART Control and Status Register B

URSEL	UMSEL	UPM1	UPM0	USBS	UCSZ1	UCSZ0	UCPOL

URSEL (Bit 7): Register Select
This bit selects to access either the UCSRC or the UBRRH register and will be discussed more in this section.

UMSEL (Bit 6): USART Mode Select
This bit selects to operate in either the asynchronous or synchronous mode of operation.

0 = Asynchronous operation
1 = Synchronous operation

UPM1:0 (Bit 5:4): Parity Mode
These bits disable or enable and set the type of parity generation and check.

00 = Disabled
01 = Reserved
10 = Even Parity
11 = Odd Parity

USBS (Bit 3): Stop Bit Select
This bit selects the number of stop bits to be transmitted.

0 = 1 bit
1 = 2 bits

UCSZ1:0 (Bit 2:1): Character Size
These bits combined with the UCSZ2 bit in UCSRB set the character size in a frame and will be discussed more in this section.

UCPOL (Bit 2): Clock Polarity
This bit is used for synchronous mode only and will not be covered in this section.

Figure 11-15. UCSRC: USART Control and Status Register C

Table 11-5: Values of UCSZ2:0 for Different Character Sizes

UCSZ2	UCSZ1	UCSZ0	Character Size
0	0	0	5
0	0	1	6
0	1	0	7
0	1	1	8
1	1	1	9

Note: Other values are reserved. Also notice that UCSZ0 and UCSZ1 belong to UCSRC and UCSZ2 belongs to UCSRB

Atmega32 Interrupt Vector Table

Table 10-1: Interrupt Vector Table for the ATmega32 AVR

Interrupt	ROM Location (Hex)
Reset	0000
External Interrupt request 0	0002
External Interrupt request 1	0004
External Interrupt request 2	0006
Time/Counter2 Compare Match	0008
Time/Counter2 Overflow	000A
Time/Counter1 Capture Event	000C
Time/Counter1 Compare Match A	000E
Time/Counter1 Compare Match B	0010
Time/Counter1 Overflow	0012
Time/Counter0 Compare Match	0014
Time/Counter0 Overflow	0016
SPI Transfer complete	0018
USART, Receive complete	001A
USART, Data Register Empty	001C
USART, Transmit Complete	001E
ADC Conversion complete	0020
EEPROM ready	0022
Analog Comparator	0024
Two-wire Serial Interface (I2C)	0026
Store Program Memory Ready	0028

ATmega324pb Interrupt Vector Table

14.1 Interrupt Vectors in ATmega324PB

Table 14-1. Reset and Interrupt Vectors in ATmega324PB

Vector No	Program Address(2)	Source	Interrupts definition
1	0x0000(1)	RESET	External Pin, Power-on Reset, Brown-out Reset and Watchdog System Reset
2	0x0002	INT0	External Interrupt Request 0
3	0x0004	INT1	External Interrupt Request 1
4	0x0006	INT2	External Interrupt Request 2
5	0x0008	PCINT0	Pin Change Interrupt Request 0
6	0x000A	PCINT1	Pin Change Interrupt Request 1
7	0x000C	PCINT2	Pin Change Interrupt Request 2
8	0x000E	PCINT3	Pin Change Interrupt Request 3
9	0x0010	WDT	Watchdog Time-out Interrupt
10	0x0012	TIMER2_COMPA	Timer/Counter2 Compare Match A
11	0x0014	TIMER2_COMPB	Timer/Coutner2 Compare Match B
12	0x0016	TIMER2_OVF	Timer/Counter2 Overflow
13	0x0018	TIMER1_CAPT	Timer/Counter1 Capture Event
14	0x001A	TIMER1_COMPA	Timer/Counter1 Compare Match A
15	0x001C	TIMER1_COMPB	Timer/Coutner1 Compare Match B
16	0x001E	TIMER1_OVF	Timer/Counter1 Overflow
17	0x0020	TIMER0_COMPA	Timer/Counter0 Compare Match A
18	0x0022	TIMER0_COMPB	Timer/Coutner0 Compare Match B
19	0x0024	TIMER0_OVF	Timer/Counter0 Overflow
20	0x0026	SPI0_STC	SPI0 Serial Transfer Complete
21	0x0028	USART0_RX	USART0 Rx Complete
22	0x002A	USART0_UDRE	USART0, Data Register Empty

http://ww1.microchip.com/downloads/en/DeviceDoc/40001908A.pdf

Table 10-3: Interrupt Vector Name for the ATmega32/ATmega16 in WinAVR

Interrupt	Vector Name in WinAVR
External Interrupt request 0	INT0_vect
External Interrupt request 1	INT1_vect
External Interrupt request 2	INT2_vect
Time/Counter2 Compare Match	TIMER2_COMP_vect
Time/Counter2 Overflow	TIMER2_OVF_vect
Time/Counter1 Capture Event	TIMER1_CAPT_vect
Time/Counter1 Compare Match A	TIMER1_COMPA_vect
Time/Counter1 Compare Match B	TIMER1_COMPB_vect
Time/Counter1 Overflow	TIMER1_OVF_vect
Time/Counter0 Compare Match	TIMER0_COMP_vect
Time/Counter0 Overflow	TIMER0_OVF_vect
SPI Transfer complete	SPI_STC_vect
USART, Receive complete	USART0_RX_vect
USART, Data Register Empty	USART0_UDRE_vect
USART, Transmit Complete	USART0_TX_vect
ADC Conversion complete	ADC_vect
EEPROM ready	EE_RDY_vect
Analog Comparator	ANALOG_COMP_vect
Two-wire Serial Interface	TWI_vect
Store Program Memory Ready	SPM_RDY_vect